

AJL Viva Q&A; Reference Guide

Study companion covering SOLID Principles, Streams API, and Spring Boot Microservices. Compiled for quick revision and oral presentation.

Q1) SOLID Principles: Design a Notification System

Question: Create a notification system that sends alerts via Email, SMS, and Push. It should be easy to add new channels (like Slack) without changing existing code.

Core Concepts & SOLID Alignment:

- **Single Responsibility Principle (SRP):** Each sender class (EmailSender, SmsSender, PushSender) has only one reason to change, which is the modification of its respective channel configuration or communication mechanism.
- **Dependency Inversion Principle (DIP):** The high-level module (NotificationService) does not depend on low-level concrete implementations (like EmailSender). Instead, it depends on the abstraction NotificationSender. We pass the interface implementation via the constructor (Constructor Injection).
- **Open-Closed Principle (OCP):** The system is open for extension but closed for modification. We can easily add a new channel like SlackSender by implementing the NotificationSender interface, without altering the existing NotificationService or other classes.

Java Implementation:

```
// Interface representing the abstract channel
interface NotificationSender {
    void send(String message, String recipient);
}

// Concrete Implementations of channels (each has a single responsibility)
class EmailSender implements NotificationSender {
    @Override
    public void send(String message, String recipient) {
        System.out.println("Email sent to " + recipient + " | Message: " + message);
    }
}

class SmsSender implements NotificationSender {
    @Override
    public void send(String message, String recipient) {
        System.out.println("SMS sent to " + recipient + " | Message: " + message);
    }
}

class PushSender implements NotificationSender {
    @Override
    public void send(String message, String recipient) {
        System.out.println("Push notification sent to " + recipient + " | Message: " + message);
    }
}

// NotificationService depends on abstraction (DIP)
class NotificationService {
    private final NotificationSender sender;

    // Constructor injection of the dependency
    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }

    public void alert(String message, String recipient) {
        sender.send(message, recipient);
    }
}

// Easy to extend for Slack (OCP)
class SlackSender implements NotificationSender {
    @Override
    public void send(String message, String recipient) {
        System.out.println("Slack message sent to channel #" + recipient + " | Message: " + message);
    }
}
```

Test / Demonstration Block:

```
public class NotificationDemo {
    public static void main(String[] args) {
        String msg = "Server is down!";

        // Testing all three channels using polymorphism
        NotificationService emailService = new NotificationService(new EmailSender());
        emailService.alert(msg, "admin@company.com");

        NotificationService smsService = new NotificationService(new SmsSender());
        smsService.alert(msg, "+1234567890");

        NotificationService pushService = new NotificationService(new PushSender());
        pushService.alert(msg, "Device_ID_9876");

        // Testing the extension (Slack)
        NotificationService slackService = new NotificationService(new SlackSender());
        slackService.alert(msg, "alerts-channel");
    }
}
```

Q2) SOLID Principles: Shape Area Calculator

Question: Build a shape area calculator that can handle new shapes without modifying the calculator class.

Core Concepts & SOLID Alignment:

- **Open-Closed Principle (OCP):** The AreaCalculator class is closed for modification. Its calculateTotalArea method accepts a list of Shape objects and uses runtime polymorphism to evaluate individual areas. We can add Pentagon or any future shape without editing a single line of the calculator class.
- **Polymorphism:** By defining an abstract method area() in the Shape interface, we hide specific calculation logic (like πr^2 for circles or $0.5 * b * h$ for triangles) behind a common method contract.

Java Implementation:

```
import java.util.List;

// Shape abstraction interface
interface Shape {
    double area();
}

// Concrete Shape classes
class Circle implements Shape {
    private final double radius;
    public Circle(double radius) { this.radius = radius; }
    @Override public double area() { return Math.PI * radius * radius; }
}

class Rectangle implements Shape {
    private final double width, height;
    public Rectangle(double w, double h) { this.width = w; this.height = h; }
    @Override public double area() { return width * height; }
}

class Triangle implements Shape {
    private final double base, height;
    public Triangle(double b, double h) { this.base = b; this.height = h; }
    @Override public double area() { return 0.5 * base * height; }
}

// AreaCalculator is closed for modification
class AreaCalculator {
    public double calculateTotalArea(List<Shape> shapes) {
        double total = 0;
        for (Shape shape : shapes) {
            total += shape.area(); // Polymorphic method call
        }
        return total;
    }
}

// Extension: Adding Pentagon without modifying AreaCalculator class
class Pentagon implements Shape {
    private final double side;
    public Pentagon(double side) { this.side = side; }
    @Override
    public double area() {
        // Area of a regular pentagon = 0.25 * sqrt(5 * (5 + 2 * sqrt(5))) * s^2
        return 0.25 * Math.sqrt(5 * (5 + 2 * Math.sqrt(5))) * side * side;
    }
}
```

Test / Demonstration Block:

```
public class ShapeDemo {
    public static void main(String[] args) {
        List<Shape> shapes = List.of(
            new Circle(5),          // Area ~ 78.54
            new Rectangle(4, 5),    // Area = 20
            new Triangle(3, 4),     // Area = 6
            new Pentagon(4)         // Area ~ 27.52
        );

        AreaCalculator calculator = new AreaCalculator();
        double totalArea = calculator.calculateTotalArea(shapes);
        System.out.printf("Total Area of mixed shapes: %.2f\n", totalArea);
    }
}
```

Q3) Streams API: Employee Report Generator

Question: Given a list of employees (Name, Department, Salary, Year Joined), produce a report containing average salary per department, highest paid employee in each department, and a comma-separated list of names of employees who joined after 2019 sorted alphabetically.

Stream API Operations Used:

- `Collectors.groupingBy()`: Groups elements by a classification function (department).
- `Collectors.averagingDouble()`: Calculates the average of a double-valued property of objects.
- `Collectors.collectingAndThen()`: Adapts a collector to perform an additional finishing transformation (extracting values from Optionals).
- `Collectors.maxBy()`: Finds the maximum element according to a comparator (salary).
- `filter()`, `map()`, `sorted()`, `Collectors.joining(", ")`: Processes pipelines for filtering by year, extracting names, sorting alphabetically, and stitching into a comma-separated string.

Java Implementation:

```
import java.util.*;
import java.util.stream.Collectors;

class Employee {
    String name, department;
    double salary;
    int yearJoined;

    public Employee(String name, String dept, double sal, int year) {
        this.name = name; this.department = dept; this.salary = sal; this.yearJoined = year;
    }
    public String getName() { return name; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }
    public int getYearJoined() { return yearJoined; }

    @Override
    public String toString() { return name + " (" + salary + ")"; }
}

public class EmployeeReportGenerator {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Alice", "Engineering", 95000, 2018),
            new Employee("Bob", "Marketing", 60000, 2021),
            new Employee("Carol", "Engineering", 112000, 2020),
            new Employee("Dave", "HR", 52000, 2019),
            new Employee("Eve", "Marketing", 78000, 2022),
            new Employee("Frank", "Engineering", 88000, 2021)
        );

        // 1. Average salary per department
        Map<String, Double> avgSalaryPerDept = employees.stream()
            .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.averagingDouble(Employee::getSalary)
            ));
        System.out.println("Average Salary Per Department: " + avgSalaryPerDept);

        // 2. Highest paid employee in each department
        Map<String, Employee> highestPaidPerDept = employees.stream()
            .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.collectingAndThen(
                    Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary)),
                    Optional::get
                )
            ));
        System.out.println("Highest Paid in Each Department: " + highestPaidPerDept);

        // 3. Comma-separated list of names of employees who joined after 2019, sorted alphabetically
        String joinedAfter2019 = employees.stream()
            .filter(e -> e.getYearJoined() > 2019)
            .map(Employee::getName)
            .sorted()
            .collect(Collectors.joining(", "));
        System.out.println("Employees who joined after 2019: " + joinedAfter2019);
    }
}
```

Q4) Streams API: File Word Frequency Counter

Question: Read a text file and use the Streams API to: 1. Count the frequency of each word (case-insensitive, ignore punctuation), 2. Print top 10 most frequent words, 3. Find words appearing exactly once, and 4. Count unique words longer than 6 characters.

Key Stream Operations:

- `Files.lines()`: Reads lines from a file as a `Stream<String>` in a memory-efficient, lazy-loaded fashion.
- `flatMap()`: Flattens a stream of arrays/streams (individual lines split into arrays of words) into a single continuous stream of elements.
- `Collectors.toMap(..., Integer::sum)`: Aggregates list contents into maps while resolving key collisions by executing a merge function (counting frequencies).

Java Implementation:

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class WordFrequencyCounter {
    public static void main(String[] args) {
        String filePath = "sample.txt"; // Target test text file

        try (Stream<String> lines = Files.lines(Paths.get(filePath))) {
            // Process lines into a frequency map
            Map<String, Long> wordFrequencies = lines
                .flatMap(line -> Arrays.stream(line.split("\\W+"))) // Split by non-word characters
                .map(String::toLowerCase) // Convert to lowercase (case-insensitive)
                .filter(word -> !word.isEmpty()) // Exclude empty tokens
                .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));

            // 1. Print all frequencies (brief display or total size)
            System.out.println("Total Unique Words Found: " + wordFrequencies.size());

            // 2. Print top 10 most frequent words
            System.out.println("\n--- Top 10 Most Frequent Words ---");
            wordFrequencies.entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
                .limit(10)
                .forEach(entry -> System.out.println(entry.getKey() + ": " + entry.getValue()));

            // 3. Find all words that appear exactly once
            List<String> onceAppearingWords = wordFrequencies.entrySet().stream()
                .filter(entry -> entry.getValue() == 1)
                .map(Map.Entry::getKey)
                .collect(Collectors.toList());
            System.out.println("\nWords appearing exactly once: " + onceAppearingWords);

            // 4. Count how many unique words are longer than 6 characters
            long longWordsCount = wordFrequencies.keySet().stream()
                .filter(word -> word.length() > 6)
                .count();
            System.out.println("\nUnique words longer than 6 chars: " + longWordsCount);

        } catch (IOException e) {
            System.err.println("Error reading file: " + e.getMessage());
        }
    }
}
```

Q5) Microservices: Student & Course Registration

Question: Build two Spring Boot microservices that talk to each other: StudentService (port 8081) and CourseService (port 8082). CourseService calls StudentService to verify student existence during enrollment, handling service downtime and bad IDs gracefully.

Core Concepts & Architectural Workflow:

- **Inter-service Communication:** CourseService makes an HTTP GET request to `http://localhost:8081/students/{id}` using Spring's RestTemplate or WebClient.
- **Resilience and Fallbacks:** When the StudentService is down, the RestTemplate calls throw a ResourceAccessException. We wrap the HTTP request in a try-catch block to return a clean, friendly error message instead of letting Spring dump a stack trace to the user.
- **Client Error Handling:** If the student does not exist, StudentService returns HTTP 404. CourseService catches HttpClientErrorException.NotFound to return a structured JSON response (e.g. 'Cannot enroll: Student ID not found').

StudentService Controller (Port 8081):

```
@RestController
@RequestMapping("/students")
public class StudentController {
    // In-memory storage mapping studentId -> name
    private final Map<String, String> studentDb = new ConcurrentHashMap<>();

    @PostMapping
    public ResponseEntity<String> registerStudent(@RequestBody Student student) {
        studentDb.put(student.getId(), student.getName());
        return ResponseEntity.ok("Student registered successfully.");
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudent(@PathVariable String id) {
        if (!studentDb.containsKey(id)) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).build(); // 404
        }
        return ResponseEntity.ok(new Student(id, studentDb.get(id)));
    }
}
```

CourseService Controller & Inter-service Communication (Port 8082):

```
@RestController
public class CourseController {
    private final RestTemplate restTemplate = new RestTemplate();
    private final String STUDENT_SERVICE_URL = "http://localhost:8081/students/";
    private final Map<String, List<String>> courseEnrollments = new ConcurrentHashMap<>(); // courseId -> studentIds

    @PostMapping("/enroll")
    public ResponseEntity<String> enrollStudent(@RequestParam String courseId, @RequestParam String studentId) {
        // Step 1: Call StudentService to verify student exists
        try {
            String targetUrl = STUDENT_SERVICE_URL + studentId;
            // Calls StudentService. Returns student details or throws 404
            restTemplate.getForObject(targetUrl, Object.class);
        }
        catch (HttpClientErrorException.NotFound e) {
            // Graceful handling of invalid Student ID
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body("Enrollment Failed: Student ID " + studentId + " does not exist.");
        }
        catch (ResourceAccessException e) {
            // Graceful handling of StudentService downtime
            return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
                .body("Enrollment Failed: Student Verification Service is currently offline. Please try again later.");
        }

        // Step 2: Proceed with enrollment if no exception was thrown
        courseEnrollments.computeIfAbsent(courseId, k -> new ArrayList<>()).add(studentId);
        return ResponseEntity.ok("Student " + studentId + " enrolled successfully in " + courseId + ".");
    }
}
```